Module I:
Agentic AI and LLM Applications
Software Installation and Setup Guide

Development Environment: Python 3.12, Spyder, Ollama, LangChain, LangGraph, Flask, and Dash

Default LLM: Qwen3.5:4b

1. Overview

This guide provides the software installation requirements for the four-day Agentic AI training program. Participants will develop AI applications using local large language models, retrieval-augmented generation (RAG), tools, stateful workflows, multi-agent systems, human approval, evaluation, guardrails, and deployment. The training uses Python and open-source software. Participants should complete the required installation steps **before the first training day**.

Software requirements by training session

Session	Topic	Main software and packages
1	Introduction to Agentic AI	Office Apps
2	Deep Learning, Transformers, and LLMs	TensorFlow, Keras, KerasHub
3	Local LLMs, Prompting, and Structured Outputs	Ollama, Qwen, Pydantic
4	Embeddings and RAG	Sentence Transformers, LangChain, PyPDF, BM25
5	Agent Orchestration, Tools, State, Memory, and MCP	LangChain, LangGraph, SQLite
6	Stateful Multi-Agent Systems and Human Oversight	LangGraph, workflow checkpoints
7	Agentic AI Assurance	scikit-learn, pytest, pandas, matplotlib
8	LLMOps and Deployment	Flask, Dash, Plotly, Waitress

The group capstone project will integrate these technologies into a complete Agentic AI application.

2. Recommended Laptop Specifications

Component	Recommended specification
Operating system	Windows 11 (64-bit), recent macOS, or Linux
Processor	Recent 8-core CPU or better
RAM	16 GB minimum for shared-server use; 24–32 GB recommended for local LLMs
Storage	At least 30 GB available for shared-server use; 80 GB recommended for local models and datasets
GPU	Optional NVIDIA GPU with 8 GB VRAM or more
Apple Silicon	M1 or newer; 16–32 GB unified memory recommended for local inference
Internet	Required for installation, downloads, and selected external integrations
Browser	Google Chrome, Microsoft Edge, Firefox, or another modern browser

A GPU is not required for the Python development, RAG, orchestration, evaluation, or Flask/Dash exercises.

3. Install Anaconda or Miniforge

Install one of the following Python distributions:

- **Anaconda Distribution:** https://www.anaconda.com/download
- **Miniforge:** https://github.com/conda-forge/miniforge

Anaconda provides Python, Conda environment management, and Anaconda Navigator. Miniforge is a lighter alternative that uses the conda-forge package repository.

Either distribution is suitable for the workshop. Only one is needed. After installation, open **Anaconda Prompt** on Windows or **Terminal** on macOS/Linux and verify:

```
conda --version
```

The command should display the installed Conda version.

4. Create the Python Environment and Configure Spyder

The training uses **Python 3.12** and the Spyder IDE. Participants will create a dedicated Conda environment named *agentic_ai* for all Python packages used throughout the four-day training.

Participants who already have Spyder installed through Anaconda may continue using their existing installation.

Step 4.1 — Install or Launch Spyder

Option A — If Spyder is already installed through Anaconda

If you already have a working Spyder installation through Anaconda or Anaconda Navigator, **you do not need to install Spyder again**.

Launch Spyder using Anaconda Navigator or from the Windows Start menu. Alternatively, open Anaconda Prompt and run:

```
conda activate base
spyder
```

If Spyder opens successfully, proceed directly to **Step 4.2**.

Option B — If Spyder is not yet installed

Open Anaconda Prompt (Windows) or Terminal (macOS/Linux), then run:

```
conda create -n spyder_env -c conda-forge spyder -y
```

Launch Spyder:

```
conda activate spyder_env
spyder
```

Spyder should open successfully.

This installs Spyder in a separate environment to avoid conflicts with the training's Python libraries.

Step 4.2 — Create the Training Environment

Regardless of whether Spyder is already installed, create a separate Python environment for the workshop. Open Anaconda Prompt or Terminal and run:

```
conda create -n agentic_ai python=3.12 -y
conda activate agentic_ai
```

Install the Spyder kernel support package:

```
conda install -c conda-forge spyder-kernels -y
```

> **Note:** If Spyder subsequently reports that a different spyder-kernels version is required, follow the version-specific installation instructions displayed by Spyder. Different Spyder versions may require different kernel versions.
>
> Update pip:
>
> ```
> python -m pip install --upgrade pip
> ```
>
> Verify the Python version:
>
> ```
> python --version
> ```
>
> Expected result:
>
> ```
> Python 3.12.x
> ```

The *agentic_ai* environment will contain the Python packages used throughout Sessions 1–8 and the group capstone project.

Important: Participants who already have Spyder installed should not reinstall it inside *agentic_ai*. Only spyder-kernels and the required training packages need to be installed in this environment.

Step 4.3 — Connect Spyder to the Training Environment

Spyder must be configured to execute Python code using *agentic_ai* instead of the default Anaconda environment.

Option A — Select the Environment in Spyder

1. Open Spyder.
2. From the menu, select **Consoles → New console in environment**.
3. Select *agentic_ai* from the available environments.

4. A new IPython console should open using the training environment.

Option B — Manually Configure the Python Interpreter

If *agentic_ai* does not appear in the list:

1. Open Anaconda Prompt or Terminal and run:
   ```
   conda activate agentic_ai
   python -c "import sys; print(sys.executable)"
   ```
2. Copy the Python executable path displayed.
3. In Spyder, open **Tools → Preferences → Python Interpreter**.
4. Select the option to use a custom Python interpreter or select an existing environment.
5. Paste or browse to the Python executable path obtained in Step 1.
6. Apply the settings and restart the IPython kernel.

The location and wording of these settings may vary slightly depending on the installed Spyder version.

Step 4.4 — Verify the Python Environment in Spyder

In the Spyder IPython console, run:

```
#%% VERIFY PYTHON ENVIRONMENT
import sys
print("Python version:", sys.version.split()[0])
print("Python executable:", sys.executable)
```

The output should indicate Python 3.12 and an executable path containing *agentic_ai*. For example, on Windows:

```
Python version: 3.12.x
Python executable: C:\Users\...\anaconda3\envs\agentic_ai\python.exe
```

On macOS/Linux, the executable path will typically contain:

```
.../envs/agentic_ai/bin/python
```

If the executable path points to the base or spyder_env environment instead, repeat Step 4.3 to select the correct Python interpreter. **All workshop Python scripts must execute using the *agentic_ai* environment.** Spyder itself may run from the Anaconda base environment or a separate *spyder_env*. After successful verification, proceed to Section 5 to install the Python packages required for the training.

5. Install the Required Python Packages

Activate the training environment:

```
conda activate agentic_ai
```

Run the following commands in Anaconda Prompt or Terminal, not in the Spyder Python console.

Step 5.1 — Core Python and Data Science

```
python -m pip install numpy pandas matplotlib scikit-learn requests pydantic python-dotenv
python -m pip install pytest
```

These packages support tabular data processing, evaluation metrics, plotting, API requests, output validation, and automated tests.

Step 5.2 — Deep Learning and Transformers

```
python -m pip install tensorflow keras keras-hub
```

These packages are used in Session 2 for neural network and pretrained transformer demonstrations. On native Windows, modern TensorFlow will generally use the CPU. Windows GPU acceleration through WSL2 is an advanced option and is not required for the workshop.

Step 5.3 — Local LLM Integration

```
python -m pip install ollama
```

The Ollama Python library allows Python programs to communicate with an Ollama model server.

Important: Installing the Python ollama library does not install the Ollama application or download the language model. You will have to do it in Step 6.

Step 5.4 — Embeddings and RAG

```
python -m pip install sentence-transformers pypdf rank-bm25
```

These packages provide semantic embeddings, PDF extraction, and keyword-based document retrieval.

Step 5.5 — LangChain and Agent Orchestration

```
python -m pip install langchain langchain-core langchain-community langchain-text-splitters
python -m pip install langchain-huggingface langchain-ollama langgraph
python -m pip install langgraph-checkpoint-sqlite
```

These packages support LLM integration, RAG workflows, tool calling, stateful agent graphs, multi-agent coordination, and SQLite-backed workflow checkpoints.

Step 5.6 — Flask, Dash, and Deployment

```
python -m pip install flask dash plotly waitress
```

These packages support building REST APIs, creating interactive browser interfaces and monitoring dashboards, and serving the application through a production-oriented WSGI server.

Step 5.7 — Check package compatibility

Run:

```
python -m pip check
```

If successful, the command should report that no broken package requirements were found.

This verifies declared package dependency compatibility. It does not replace the runtime tests in Section 9.

6. Install Ollama and Download the Required Models

Ollama is the runtime used to host the local large language models.

Download: https://ollama.com/download

Install the application for your operating system.

Step 6.1 — Verify Ollama

Open a new terminal:

```
ollama -version
```

Make sure Ollama is running. The application normally starts its background service automatically on Windows and macOS.

Step 6.2 — Download the default LLMs

The primary models for the workshop are **Qwen3.5:4b and Gemma 3**.

Run:

```
ollama pull qwen3.5:4b
ollama pull gemma3:4b
```

Step 6.3 — Download the embedding model

Some RAG and DV Helpdesk exercises use Ollama to generate text embeddings.

Run:

```
ollama pull nomic-embed-text
```

Step 6.4 — Verify the models

Run:

```
ollama list
```

Confirm that the required models appear.

Test the main model:

```
ollama run qwen3.5:4b
```

Enter:

```
Reply only with: SESSION READY
```

Confirm that the model responds, then type `/bye` to exit.

7. Download the Sentence Transformer Models

Some RAG exercises use Sentence-BERT (SBERT) rather than Ollama-based embeddings.
These models are downloaded automatically by the sentence-transformers package when first used.
To download them before the workshop, run the following code in Spyder:

```
#%% DOWNLOAD RAG EMBEDDING MODELS
from sentence_transformers import SentenceTransformer
SentenceTransformer(
    "sentence-transformers/all-MiniLM-L6-v2"
)
SentenceTransformer(
    "sentence-transformers/multi-qa-MiniLM-L6-cos-v1"
)
print("SBERT EMBEDDING MODELS READY")
```

The model files will be cached on the laptop for reuse.

8. Verify the Complete Installation

This section checks the Python environment, the main workshop packages, and the connection to Ollama.

Step 8.1 — Package verification test

Open Spyder with an IPython console using the agentic_ai environment. Copy and execute the following single Python cell:

```
#%% PACKAGE PREPARATION TEST
import sys
from importlib.metadata import version, PackageNotFoundError
packages = [
    "numpy", "pandas", "matplotlib", "scikit-learn",
    "requests", "pydantic", "pytest",
    "tensorflow", "keras", "keras-hub",
    "ollama", "sentence-transformers",
    "pypdf", "rank-bm25",
    "langchain", "langchain-core",
    "langchain-community", "langchain-text-splitters",
    "langchain-huggingface", "langchain-ollama",
    "langgraph", "langgraph-checkpoint-sqlite",
    "flask", "dash", "plotly", "waitress"
]
failed = []
print("PYTHON:", sys.version.split()[0])
for name in packages:
    try:
        version(name)
    except PackageNotFoundError:
        failed.append(name)
```

```
print("PACKAGES CHECKED:", len(packages))
print("PACKAGES MISSING:", failed if failed else "None")
print("PACKAGE TEST:", "PASS" if not failed else "INCOMPLETE")
```

This is a package-presence check.

Expected final output:

PACKAGES MISSING: None

PACKAGE TEST: PASS

Step 8.2 — Ollama connection and response test

Copy and execute the following code in Spyder:

```
#%% OLLAMA MODEL CONNECTION TEST

from ollama import Client
# Local Ollama:
OLLAMA_SERVER = "http://localhost:11434"
# For a shared server, replace with the instructor's URL.
MODEL = "qwen3.5:4b"
try:
    client = Client(host=OLLAMA_SERVER, timeout=120)
    response = client.chat(
        model=MODEL,
        messages=[
            {"role": "user",
             "content": "Reply only with: SESSION READY"}
        ],
        think=False,
        options={"num_predict": 32}
    )
    answer = response.message.content.strip()
    print("MODEL:", MODEL)
    print("MODEL RESPONSE:", answer)
    print("CONNECTION TEST: PASS")
except Exception as error:
    print("CONNECTION TEST: FAILED")
    print("ERROR:", error)
```

The expected model response is:

SESSION READY

Minor variation in the model's wording is acceptable if a valid response is returned.

The connection test should display CONNECTION TEST: PASS.

Step 8.3 — Verify the main development libraries

Run:

```
#%% FRAMEWORK RUNTIME TEST

import tensorflow as tf
```

```
import keras_hub
from sentence_transformers import SentenceTransformer
from langchain_ollama import ChatOllama
from langgraph.graph import StateGraph
from flask import Flask
from dash import Dash

print("TensorFlow:", tf.__version__)
print("KerasHub: OK")
print("Sentence Transformers: OK")
print("LangChain Ollama: OK")
print("LangGraph: OK")
print("Flask: OK")
print("Dash: OK")
print("FRAMEWORK TEST: PASS")
```

The first execution may take longer because TensorFlow and Sentence Transformers load their dependencies.

If the test completes without errors, the main development frameworks are available.

9. Quick Troubleshooting

Problem	Recommended action
conda command not recognized	Open Anaconda Prompt or verify the Conda installation.
Spyder cannot find an installed package	Confirm the console is using agentic_ai, not base or spyder_env.
Spyder reports a kernel-version mismatch	Install the compatible spyder-kernels version indicated by Spyder and restart the kernel.
No module named ollama	Activate agentic_ai and install the Ollama Python package.
Ollama command not recognized	Restart the terminal after installing the Ollama application.
Cannot connect to Ollama	Confirm Ollama is running and check the configured server address.
Model not found	Verify the model name using ollama list, or ask the instructor to install it on the shared server.
Sentence Transformer model download fails	Check internet access and rerun the download when connectivity is restored.
TensorFlow cannot access a Windows GPU	Use CPU execution; a GPU is not required for the workshop.
Package dependency conflicts	Run python -m pip check and report the error before changing package versions.
Python responses are slow	Reduce prompt complexity or use the instructor's shared Ollama server.
Flask/Dash port already in use	Stop the existing application or select an unused port.

Avoid installing additional packages into the Anaconda base environment or upgrading unrelated libraries during the training.

10. Final Readiness Checklist

Before Day 1, confirm the following:

- Anaconda or Miniforge is installed.
- Python 3.11 is available in the agentic_ai environment.
- Spyder launches and uses the correct environment.
- All required Python packages are installed.
- The package verification test passes.

- TensorFlow, LangChain, LangGraph, Flask, and Dash import successfully.
- Ollama is running locally or the instructor's shared server is accessible.
- Qwen3.5:4b responds to a Python request.
- The required embedding models are available.
- A writable folder is available for the group capstone project.

Participants should complete all required preparation before the first day of training.

If any installation or verification step fails, record the exact error message and report it to the instructor or technical support team on the day itself.

Official Software References

Software	Official website
Anaconda	https://www.anaconda.com/download
Miniforge	https://github.com/conda-forge/miniforge
Spyder	https://docs.spyder-ide.org/current/installation.html
Ollama	https://ollama.com/download
Ollama Python	https://github.com/ollama/ollama-python
Qwen3.5	https://ollama.com/library/qwen3.5
TensorFlow	https://www.tensorflow.org/install
KerasHub	https://keras.io/keras_hub/
LangChain	https://docs.langchain.com/
LangGraph	https://docs.langchain.com/oss/python/langgraph/overview
Flask	https://flask.palletsprojects.com/
Dash	https://dash.plotly.com/
Waitress	https://docs.pylonsproject.org/projects/waitress/